

Grids, columns, and responsive layout in Figma

Week 5, Lecture 2, Design for Builders: Ship Beautiful Products as a Founder

Autumn 2026

What we will cover

- The 12-column layout grid: how columns, gutters, and margins work together
- How to add and configure a layout grid in Figma
- Auto-layout and constraints: when to use each, and why combining them is not redundant
- Responsive breakpoints: desktop, tablet, and mobile as three separate design targets
- Macro-grid vs micro-grid: two systems that must share the same base unit

Part 1: the 12-column grid

Why 12 columns

12 divides evenly into 2, 3, 4, and 6. That gives you four natural column spans:

- Full width (12 of 12)
- Half width (6 of 12): two-column layout
- Third width (4 of 12): three-column layout
- Quarter width (3 of 12): four-column layout

No other number below 12 divides as cleanly. 10 columns cannot produce thirds. 8 columns cannot produce thirds or quarters that look proportional.

Columns, gutters, and margins

Three values define a layout grid.

Columns: the number of vertical divisions (almost always 12).

Gutters: the fixed-width gap between adjacent columns. Gutters do not expand when the viewport widens; only the column width changes.

Margins: the space from the frame edge to the first and last column. Margins also stay fixed or have a defined minimum.

Column width is derived, not chosen: $(\text{frame width} - 2 * \text{margin} - 11 * \text{gutter}) / 12$.

Typical values by breakpoint

```
Desktop 1440px:  margin 80px, gutter 24px → column ~72px  
Tablet  768px:  margin 32px, gutter 16px → column ~48px  
Mobile  390px:  margin 16px, gutter 12px → column ~23px
```

The margins and gutters should be on the 8-point grid. The derived column width does not need to be.

When you narrow the frame, the columns get narrower. The gutters and margins stay the same.

Setting up a layout grid in Figma

1. Select your top-level frame (1440px, 768px, or 390px wide).
2. In the right panel, click the "+" next to "Layout grid."
3. Switch the grid type from "Grid" to "Columns."
4. Set Count: 12. Set Gutter and Margin to your target values. Set Type: "Stretch."

"Stretch" means the columns fill the frame width automatically as you resize.

Turn the grid on and off with Shift+G. It is a checking overlay, not an enforcement tool. You still type the correct values yourself.

Part 2: constraints and auto-layout

The question everyone asks

You already know auto-layout from Week 1. In this lecture, we add constraints.

Both handle responsive behavior. They are not the same.

- Auto-layout: how a frame distributes its **children**
- Constraints: how a frame positions itself relative to its **parent**

Most layouts need both. The confusion comes from using one where the other belongs.

Auto-layout recap

Auto-layout controls three things inside a frame:

- **Direction:** row or column (or wrap, for grid-like flow)
- **Padding:** space inside the frame before the first child
- **Gap:** space between children

Each child also has a resizing behavior:

- **Hug:** shrinks to fit its content
- **Fill:** expands to take up remaining space
- **Fixed:** stays at a defined size regardless of the container

Constraints: what they do

Constraints control how a frame moves and resizes when its parent is resized.

A frame can be pinned to: Left, Right, Top, Bottom, or Center (horizontal and vertical independently). It can also be set to "Scale" (moves proportionally with the parent) or "Left and right" (stretches).

Use constraints when:

- A frame lives inside a non-auto-layout parent
- You need a frame to stay pinned to a corner while the parent resizes
- You are building a sticky header or a fixed-position overlay

When to use which

Auto-layout: when you want children to rearrange in response to content.

Constraints: when you want a frame to pin or stretch in response to its parent's size changing.

A common layout:

```
Page frame (no auto-layout, a fixed canvas)
  Header frame (constraints: left+right, top)
    ← auto-layout handles the header's internal row of items
  Section frame (constraints: left+right, top)
    ← auto-layout handles the section's column of rows
    ← each row auto-layout handles the column layout
```

Part 3: responsive breakpoints

Three frames, not one

Responsive design in Figma means three separate frames at three target widths.

- **Desktop:** 1440px (sometimes 1280px for laptops)
- **Tablet:** 768px
- **Mobile:** 390px (iPhone 14 logical width)

Design desktop first, clean up components, then adapt to tablet and mobile. Components with children set to Fill width adapt with less manual adjustment.

Do not try to represent all breakpoints in one frame. They are different design targets, not different states of the same frame.

What changes between breakpoints

From desktop to tablet:

- Column grids often halve: 3 columns becomes 2 or 1
- Navigation collapses from a top bar to a collapsed menu
- Margins and gutters narrow
- Type scale may drop one step

From tablet to mobile:

- Most layouts become single-column
- Horizontal padding narrows to 16px or 12px
- Touch targets must be at least 44px tall
- Horizontal navigation becomes a bottom tab bar or hamburger

Components that survive breakpoints

A component built with Fill width children adapts automatically when you place it in a narrower frame.

A component with Fixed width children must be manually adjusted at each breakpoint.

Design components for Fill first. Override to Fixed only when the design explicitly requires a fixed-size element (a logo, a fixed-width sidebar).

Figma auto-layout guide (Figma, 2023): "Set children to Fill width when you want them to stretch with the container."

Part 4: macro-grid vs micro-grid

Two grids, same base unit

Every layout operates at two scales.

Macro-grid: page-level structure. Where do the major content regions sit? How much space separates sections? What are the margins from content to viewport edge?

Micro-grid: component-level structure. What is the padding inside a card? What is the gap between items in a list? How far is a label from its input?

Both grids must use the same base unit (8px or 4px). If they do not, spacing values across the design share no common factors. The layout feels inconsistent even when each component looks correct in isolation.

The two-grid structure in practice

Macro-grid (page layout)

Section margin from viewport: 2xl (48px) or 3xl (64px)

Gap between cards in a row: lg (24px) or xl (32px)

Gap between page sections: 3xl (64px)

Micro-grid (component internals)

Card padding: md (16px)

List item gap: sm (8px)

Icon-to-label gap: xs (4px)

All values share the base unit. A 16px card padding + 24px inter-card gap + 16px card padding = 56px. Every step is on-grid.

When the two grids conflict

If a component's internal padding is 14px (not on-grid), and the outer gap is 24px, the content-to-content distance across two cards is $14 + 24 + 14 = 52\text{px}$.

52px is not a multiple of 8 or 4. The layout has no reason for that number. It cannot be expressed as a token. It cannot be adjusted systematically.

This is why the micro-grid must share the base unit with the macro-grid. One off-grid value at the component level propagates into every layout that uses the component.

A worked example: hero section

Hero frame (1440px wide, auto-layout vertical)

Padding top: 3xl (64px) ← macro: top section margin

Padding left/right: 80px (column margin)

Gap between headline and subhead: md (16px)

Gap between subhead and CTA row: lg (24px)

CTA row (auto-layout horizontal)

Gap between primary and secondary button: sm (8px)

Primary button (auto-layout)

Padding top/bottom: sm (8px) ← micro: inner button padding

Padding left/right: md (16px) ← micro: inner button padding

Every value is a named token. Every token is on the 8-point grid.

Diagnosing a broken layout

Four failure modes for spacing:

1. **Inconsistent values:** elements at the same hierarchy level use different tokens. Two adjacent cards, one at `md` padding and one at `lg` padding.
2. **Wrong proximity signal:** related elements are further apart than unrelated elements. Form label is further from its input than from the next form group.
3. **Misaligned axes:** elements that should share an edge do not. Left edges of cards vary by a pixel or two.
4. **Scale confusion:** inner spacing is larger than outer spacing. Card padding of 24px, inter-card gap of 16px.

Name the failure mode before reaching for the fix.

Take-aways

1. The 12-column grid divides into halves, thirds, and quarters. Define columns, gutters, and margins. Let column width be derived, not chosen.
2. Auto-layout controls how a frame distributes its children. Constraints control how a frame positions itself in its parent. Most real layouts need both.
3. Responsive design means three separate frames (desktop, tablet, mobile), designed in sequence. Fill-width children adapt across breakpoints; Fixed-width children must be adjusted manually.
4. The macro-grid governs page structure. The micro-grid governs component internals. Both must use the same base unit.
5. When a layout feels wrong, name the failure mode first: inconsistent values, wrong proximity, misaligned axes, or scale confusion.

"Use auto layout when you want children to rearrange in response to content changes. Use constraints when you want children to pin or stretch in response to frame size changes."

Figma, Guide to Auto Layout, 2023

What's next

- **This week, section:** Pixel-grid drill: rebuild three real product screens on an 8-point grid
- **Next week (Week 6):** Landing-page anatomy uses this week's grid and spacing as its structural foundation
- **HW2 due this week:** Type-only redesign
- **HW3 assigned next week:** Full landing page design in Figma

